МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
им. Н.Э. Баумана

Факультет “Инженерный бизнес и менеджмент”
Кафедра “Промышленная логистика (Бизнес-информатика)”



Дисциплина “Программирование на основе классов и шаблонов”

Отчет по лабораторной работе №1

Выполнил:
Студент группы ИБМ3-24Б
Сысуева Д.Д

Преподаватели:
Правдина А. Д.
Бурмистрова М.В.

Формулировка задания

Разработать программы для решения биквадратного уравнения.

Программы должны быть разработаны в виде консольного приложения на языке Python. Разработайте **две программы**:

на языке Python используя процедурную парадигму;

на языке C++ используя ООП.

Программа осуществляет ввод с клавиатуры коэффициентов **A, B, C**, вычисляет дискриминант и **ДЕЙСТВИТЕЛЬНЫЕ** корни уравнения (в зависимости от дискриминанта).

Коэффициенты A, B, C могут быть заданы в виде параметров командной строки (вариант задания параметров приведен с примером кода). Если они не заданы, то вводятся с клавиатуры в соответствии с пунктом 2. Описание работы с параметрами командной строки.

Если коэффициент A, B, C введен или задан в командной строке некорректно, то необходимо проигнорировать некорректное значение и вводить коэффициент повторно, пока коэффициент не будет введен корректно. Корректно заданный коэффициент — это коэффициент, значение которого может быть без ошибок преобразовано в действительное число.

Листинг программы

лр1.py

```
import math
import sys

def get_coef(x):
    while True:
        s = input(x)
        try:
            return float(s)
        except ValueError:
            print("Некорректное значение, повторите ввод:")


def get_roots(a, b, c):
    roots = []

    if a == 0:
        if b == 0:
            return []
```

```
        t = -c / b
        if t > 0:
            roots.append(math.sqrt(t))
            roots.append(-math.sqrt(t))
        elif t == 0:
            roots.append(0)
        return roots

    D = b*b - 4*a*c
    if D > 0:
        t1 = (-b + math.sqrt(D)) / (2*a)
        t2 = (-b - math.sqrt(D)) / (2*a)
        for t in (t1, t2):
            if t > 0:
                roots.append(math.sqrt(t))
                roots.append(-math.sqrt(t))
            elif t == 0:
                roots.append(0)
    elif D == 0:
        t = -b / (2*a)
        if t > 0:
            roots.append(math.sqrt(t))
            roots.append(-math.sqrt(t))
        elif t == 0:
            roots.append(0)
    return roots

def get_coefs_from_term():
    args = sys.argv[1:]
    coefs = [None, None, None]

    for i in range(min(3, len(args))):
        try:
            coefs[i] = float(args[i])
        except ValueError:
            if i == 0:
                name = "A"
            elif i == 1:
                name = "B"
            else:
                name = "C"
            print(f"Некорректный аргумент {name}: {args[i]}. "
                  f"Значение будет введено с клавиатуры.")
            coefs[i] = None
    return coefs
```

```
def main():
    coefs = get_coefs_from_term()

    if coefs[0] is None:
        coefs[0] = get_coef("Введите A:")
    if coefs[1] is None:
        coefs[1] = get_coef("Введите B:")
    if coefs[2] is None:
        coefs[2] = get_coef("Введите C:")

    a, b, c = coefs
    roots = get_roots(a, b, c)

    roots.sort()

    n = len(roots)
    if n == 0:
        print("Действительных корней нет")
    else:
        print(f"Найдено действительных корней: {n}")
        if n >= 1:
            print(f"  x1 = {roots[0]}")
        if n >= 2:
            print(f"  x2 = {roots[1]}")
        if n >= 3:
            print(f"  x3 = {roots[2]}")
        if n >= 4:
            print(f"  x4 = {roots[3]}")

main()
```

лр1доп.ру

```
import math
import sys

args = sys.argv[1:]
coefs = [None, None, None]

for i in range(min(3, len(args))):
    try:
        coefs[i] = float(args[i])
    except ValueError:
        if i == 0:
            name = "A"
        elif i == 1:
            name = "B"
        else:
```

```
            name = "C"
        print(f"Некорректный аргумент {name}: {args[i]}. "
              f"Значение будет введено с клавиатуры.")
        coefs[i] = None
if coefs[0] is None:
    while True:
        s = input("Введите A: ")
        try:
            coefs[0] = float(s)
            break
        except ValueError:
            print("Некорректное значение, повторите ввод:")
if coefs[1] is None:
    while True:
        s = input("Введите B: ")
        try:
            coefs[1] = float(s)
            break
        except ValueError:
            print("Некорректное значение, повторите ввод:")
if coefs[2] is None:
    while True:
        s = input("Введите C: ")
        try:
            coefs[2] = float(s)
            break
        except ValueError:
            print("Некорректное значение, повторите ввод:")
a, b, c = coefs

roots = []
if a == 0:
    if b == 0:
        roots = []
    else:
        t = -c / b
        if t > 0:
            roots.append(math.sqrt(t))
            roots.append(-math.sqrt(t))
        elif t == 0:
            roots.append(0)
else:
    D = b*b - 4*a*c
    if D > 0:
        t1 = (-b + math.sqrt(D)) / (2*a)
        t2 = (-b - math.sqrt(D)) / (2*a)
        for t in (t1, t2):
```

```
            if t > 0:
                roots.append(math.sqrt(t))
                roots.append(-math.sqrt(t))
            elif t == 0:
                roots.append(0)
    elif D == 0:
        t = -b / (2*a)
        if t > 0:
            roots.append(math.sqrt(t))
            roots.append(-math.sqrt(t))
        elif t == 0:
            roots.append(0)

roots.sort()

n = len(roots)
if n == 0:
    print("Действительных корней нет")
else:
    print(f"Найдено действительных корней: {n}")
    if n >= 1:
        print(f"  x1 = {roots[0]}")
    if n >= 2:
        print(f"  x2 = {roots[1]}")
    if n >= 3:
        print(f"  x3 = {roots[2]}")
    if n >= 4:
        print(f"  x4 = {roots[3]}")
```

лр1доп.cpp

```
#include <iostream>
#include <string>
#include <cmath>


bool isNumber(const std::string& s, double& result) {
    if (s.empty()) return false;

    size_t start = 0;
    bool negative = false;
    if (s[0] == '-' || s[0] == '+') {
        if (s[0] == '-') {
            negative = true;
        }
        start = 1;
    }
```

```
    bool hasDigit = false;
    bool hasDot = false;
    double value = 0.0;
    double fraction = 0.1;

    for (size_t i = start; i < s.size(); i++) {
        char c = s[i];
        if (c >= '0' && c <= '9') {
            hasDigit = true;
            int digit = c - '0';
            if (hasDot == false) {
                value = value * 10 + digit;
            } else {
                value = value + digit * fraction;
                fraction = fraction / 10;
            }
        } else if (c == '.' && hasDot == false) {
            hasDot = true;
        } else {
            return false;
        }
    }

    if (hasDigit == false) {
        return false;
    }

    if (negative == true) {
        result = -value;
    } else {
        result = value;
    }
    return true;
}

double getCoef(const std::string& name) {
    while (true) {
        std::cout << "Введите " << name << ": ";
        std::string s;
        std::getline(std::cin, s);
        double value;
        if (isNumber(s, value)) {
            return value;
        }
        std::cout << "Некорректное значение, повторите ввод." << std::endl;
    }
}
```

```
class Bisquare {
private:
    double a, b, c;
    double roots[4];
    int count;

public:
    Bisquare(double a1, double b1, double c1) {
        a = a1;
        b = b1;
        c = c1;
        count = 0;
    }

    void solve() {
        count = 0;

        if (a == 0) {
            if (b == 0) {
                return;
            }
            double t = -c / b;
            if (t > 0) {
                roots[count++] = std::sqrt(t);
                roots[count++] = -std::sqrt(t);
            } else if (t == 0) {
                roots[count++] = 0;
            }
            return;
        }

        double D = b*b - 4*a*c;

        if (D > 0) {
            double t1 = (-b + std::sqrt(D)) / (2*a);
            double t2 = (-b - std::sqrt(D)) / (2*a);
            double arr[2];
            arr[0] = t1;
            arr[1] = t2;
            for (int i = 0; i < 2; i++) {
                double t = arr[i];
                if (t > 0) {
                    roots[count++] = std::sqrt(t);
                    roots[count++] = -std::sqrt(t);
                } else if (t == 0) {
                    roots[count++] = 0;
```

```
            }
        }
    } else if (D == 0) {
        double t = -b / (2*a);
        if (t > 0) {
            roots[count++] = std::sqrt(t);
            roots[count++] = -std::sqrt(t);
        } else if (t == 0) {
            roots[count++] = 0;
        }
    }
}

void printRoots() {
if (count == 0) {
    std::cout << "Действительных корней нет" << std::endl;
} else {
    std::cout << "Найдено действительных корней: " << count << std::endl;
    if (count >= 1) {
        std::cout << "  x1 = " << roots[0] << std::endl;
    }
    if (count >= 2) {
        std::cout << "  x2 = " << roots[1] << std::endl;
    }
    if (count >= 3) {
        std::cout << "  x3 = " << roots[2] << std::endl;
    }
    if (count >= 4) {
        std::cout << "  x4 = " << roots[3] << std::endl;
    }
}
}
};

int main(int argc, char* arg[]) {
    double coefs[3];
    bool haveCoef[3] = {false, false, false};
    std::string names[3] = {"A", "B", "C"};

    for (int i = 1; i < argc && i <= 3; i++) {
        double value;
        if (isNumber(arg[i], value)) {
            coefs[i - 1] = value;
            haveCoef[i - 1] = true;
        } else {
            std::cout << "Некорректный аргумент " << names[i - 1] << ": " << arg[i] << ".
Значение будет введено с клавиатуры." << std::endl;
```

```
        }
    }

    for (int i = 0; i < 3; i++) {
        if (haveCoef[i] == false) {
            coefs[i] = getCoef(names[i]);
        }
    }

    Bisquare eq(coefs[0], coefs[1], coefs[2]);
    eq.solve();
    eq.printRoots();

    return 0;
}
```

Результат работы

```
dara@MacBook-Air-Dara парадигмы программирования  % python3 лр1.py 0.5 asdfghj 2
Некорректный аргумент B: asdfghj. Значение будет введено с клавиатуры.
Введите B:5
Действительных корней нет
dara@MacBook-Air-Dara парадигмы программирования  % python3 лр1.py 0.5 -2.5 2
Найдено действительных корней: 4
  x1 = -2.0
  x2 = -1.0
  x3 = 1.0
  x4 = 2.0
dara@MacBook-Air-Dara парадигмы программирования  % g++ -o lr1dop лр1доп.cpp
dara@MacBook-Air-Dara парадигмы программирования  % ./lr1dop 1 -8 16
Найдено действительных корней: 2
  x1 = 2
  x2 = -2
dara@MacBook-Air-Dara парадигмы программирования  % python3 лр1доп.py 1 -4 0
Найдено действительных корней: 3
  x1 = -2.0
  x2 = 0
  x3 = 2.0
dara@MacBook-Air-Dara парадигмы программирования  % ./lr1dop
Введите A: 0
Введите B: 3
Введите C: dfghj
Некорректное значение, повторите ввод.
Введите C: 9
Действительных корней нет
dara@MacBook-Air-Dara парадигмы программирования  %
```